



**Tecnológico
de Monterrey**

05. Procesamiento y visualización de datos

Ciencia de datos para la toma de decisiones II

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Programa de la sesión de hoy

- Terminar práctica de apertura de datos
- Repasar tidyverse y manejo de datos
- Repasar visualización en ggplot

Ejercicio de carga de datos

- 1. Vamos a terminar de cargar los datos de los archivos proporcionados el día de ayer.
- 2. Una vez cargada la información, explore el contenido y escriba en su script lo que contiene.

Procesamiento de datos

- * Es el proceso previo a la creación de análisis, modelos y visualizaciones.
- * Es un proceso a veces molesto, pero es necesario.
- * Consiste en tomar una base de datos y convertirla en otra que cumpla nuestras necesidades.
- * Siempre hay que tener en cuenta a qué queremos llegar.

Procesamiento de datos

Ejemplo de algunos procesos:

- 1) Limpieza de datos
 - 1) Convertir tipos de datos (texto -> numero)
 - 2) Eliminar registros con datos faltantes (NAs)
 - 3) Eliminar duplicados y outliers
 - 4) Estandarizar categorías (“CdMx” -> “CDMX”)
- 2) Unir tablas
- 3) Reestructurar tablas
- 4) Agregar datos por grupos
- 5) Cambiar nombres de columnas
- 6) Reclasificar grupos
- 7) Deflactar cifras
- 8) Transformar valores
- 9) Reconvertir columnas

¿Qué es el tidyverse?

El **tidyverse** es una colección de paquetes de R diseñados para su uso en ciencia de datos. Todos los paquetes miembros comparten una filosofía de diseño, estructura de datos y gramática comunes.

Para instalar el tidyverse completo, utilizamos la siguiente función:

```
install.packages("tidyverse")
```



@allison_horst

Paquetes del tidyverse



Familia tidyverse (y este amiguito -> %>%)

Paquetes del tidyverse



- La pipa es el elemento central del tidyverso y su misión es la de **concatenar funciones**. Opera vinculando los objetos del lado izquierdo con las funciones del lado derecho.
- Para escribirlo se utiliza:
 - **MAC**: command + shift + m
 - **Windows**: control + shift + m

Operador pipa (%>%)

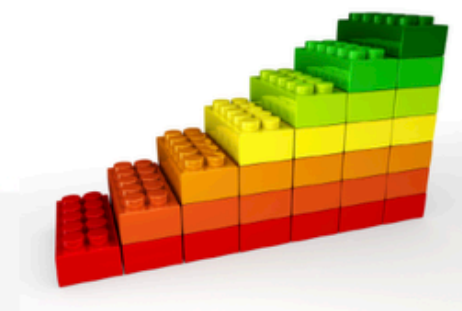


De manera coloquial, el operador pipa se debe leer como un **"y después"**. Por ejemplo, tomemos el objeto Juve.

* Con pipa:

```
# Un objeto Juve
Juve %>%
  despertarse(hora = "5:00 a.m.") %>% # Juve despierto (y despues...)
  levantarse(hora = "5:10 a.m.") %>% # Juve levantado (y despues...)
  bañarse() %>% # Juve despierto, levantado y bañado (y despues...)
  vestirse() %>% # Juve ahira vestido (y despues...)
  desayunar() %>% # Juve ahora desayunado (y despues...)
  salir_al_trabajo() # Juve rumbo al CIDE
```

Legos



Capas de cebolla



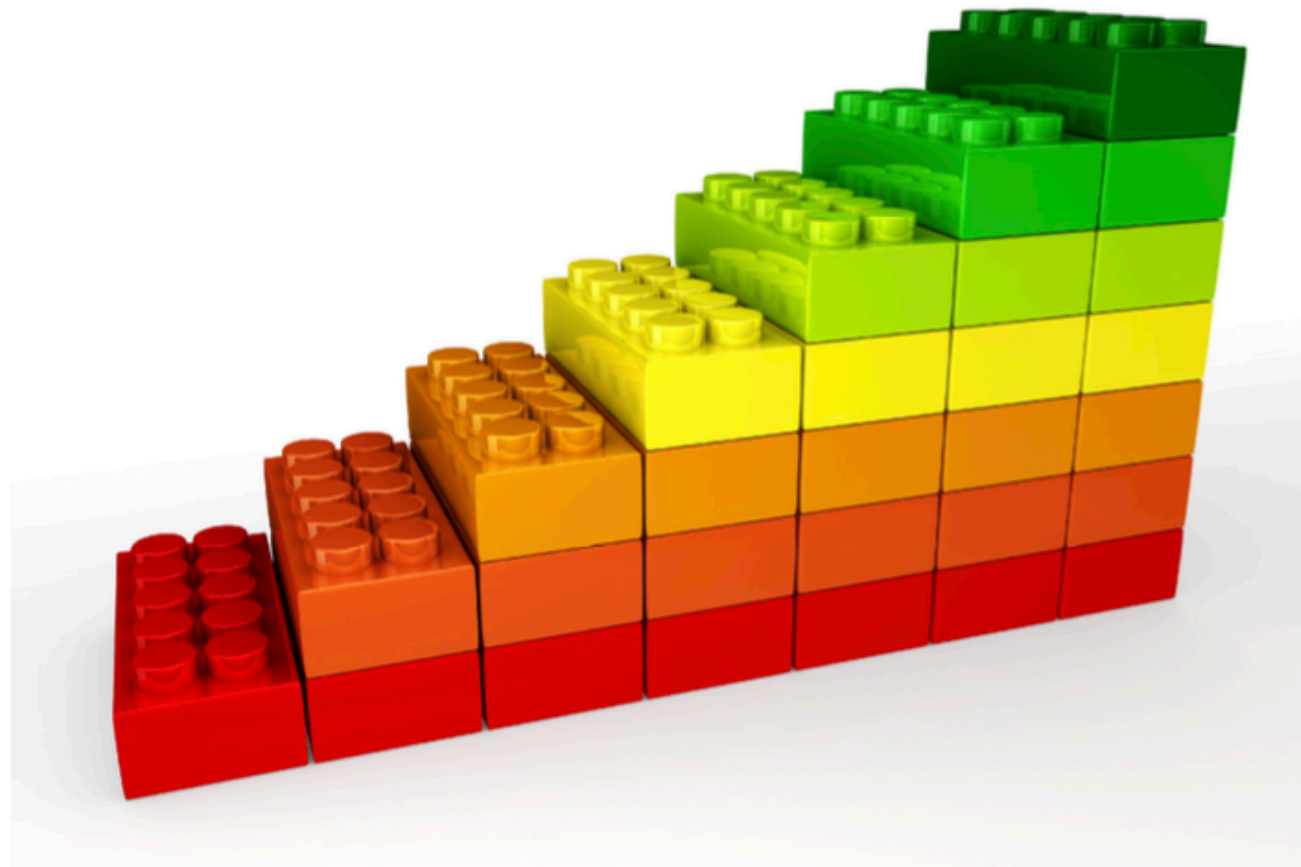
* Sin pipa:

```
# Sin la pipa
salir_al_trabajo(desayunar(vestirse(bañarse(levantarse(despertarse(Juve, hora = "5:00 a.m."), hora = "5:10 a.m.")))))
```

Operador pipa (%>%)



El operador %>% toma un objeto (en este caso Juve) y le va aplicando funciones (en este caso, todas las relativas a su rutina diaria matutina) de manera secuencial. Esto se va haciendo a manera de la unión de bloques de lego para armar una torre.



Operador pipa (%>%)



Tip 1 Si un día te sale un error y no sabes en que parte del pipeline se encuentra, corre una por una los bloques de código hasta que encuentres el error.

Tip 2 La estructura de trabajo con pipelines nos permite quitar (o silenciar, convirtiendo código en comentario) pedazos de código de manera muy fácil para hacer pruebas. Por ejemplo:

```
# Un objeto Juve
Juve %>%
  despertarse(hora = "8:00 a.m.") %>% # Juve despierto
  levantarse(hora = "8:10 a.m.") %>% # Juve levantado
#   bañarse() %>% # Juve despierto, levantado y bañado
  vestirse() %>% # Juve vestido
#   desayunar() %>% # Juve desayunado
  salir_al_trabajo() # Juve rumbo al CIDE
```


Operador pipa (%>%)



* Sin pipa:

```
# Declaramos el vector numérico `x`  
x <- c(0.109, 0.359, 0.63, 0.996, 0.515, 0.142, 0.017, 0.829, 0.907)
```

```
# Obtenemos el logaritmo de X, sacamos diferencias de un periodo, al resultado le  
sacamos la exponencial y después redondeamos el resultado.  
round(exp(diff(log(x))), 1)
```

```
## [1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

* Con pipa:

```
# La pipa ya viene importada en el tidyverse
```

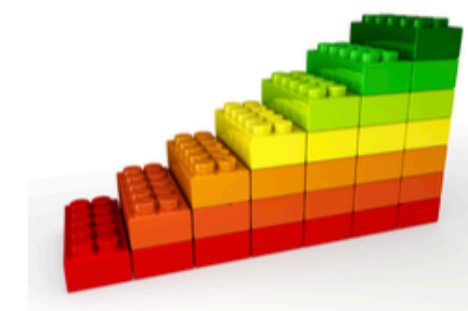
```
# Hacemos las mismas operaciones que arriba:  
x %>% log() %>%  
  diff() %>%  
  exp() %>%  
  round(1)
```

```
## [1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

Capas de cebolla



Legos



Verbos del tidyverse



Las principales funciones del **tidyverse** para tratar con datos son las siguientes:

- **filter()**, para filtrar renglones,
- **select()**, para seleccionar columnas,
- **mutate()**, para generar nuevas columnas,
- **arrange()**, para ordenar los renglones en función de una variable,
- **group_by()**, para generar grupos o clusters de renglones,
- **summarise()**, para generar cálculos con las agrupaciones del **group_by()**

filter()

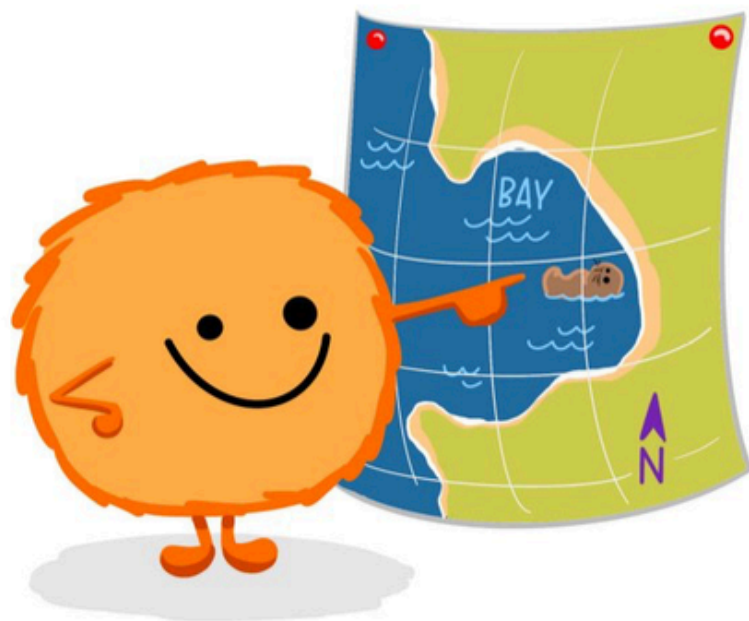


dplyr::filter()

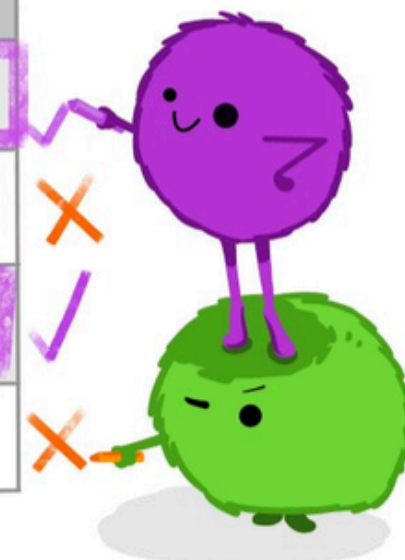
KEEP ROWS THAT
satisfy
your **CONDITIONS**

keep rows from... this data... ONLY IF... type is "otter" AND site is "bay"

```
filter(df, type == "otter" & site == "bay")
```



type	food	site
otter	urchin	bay
shark	seal	channel
otter	abalone	bay
otter	crab	wharf

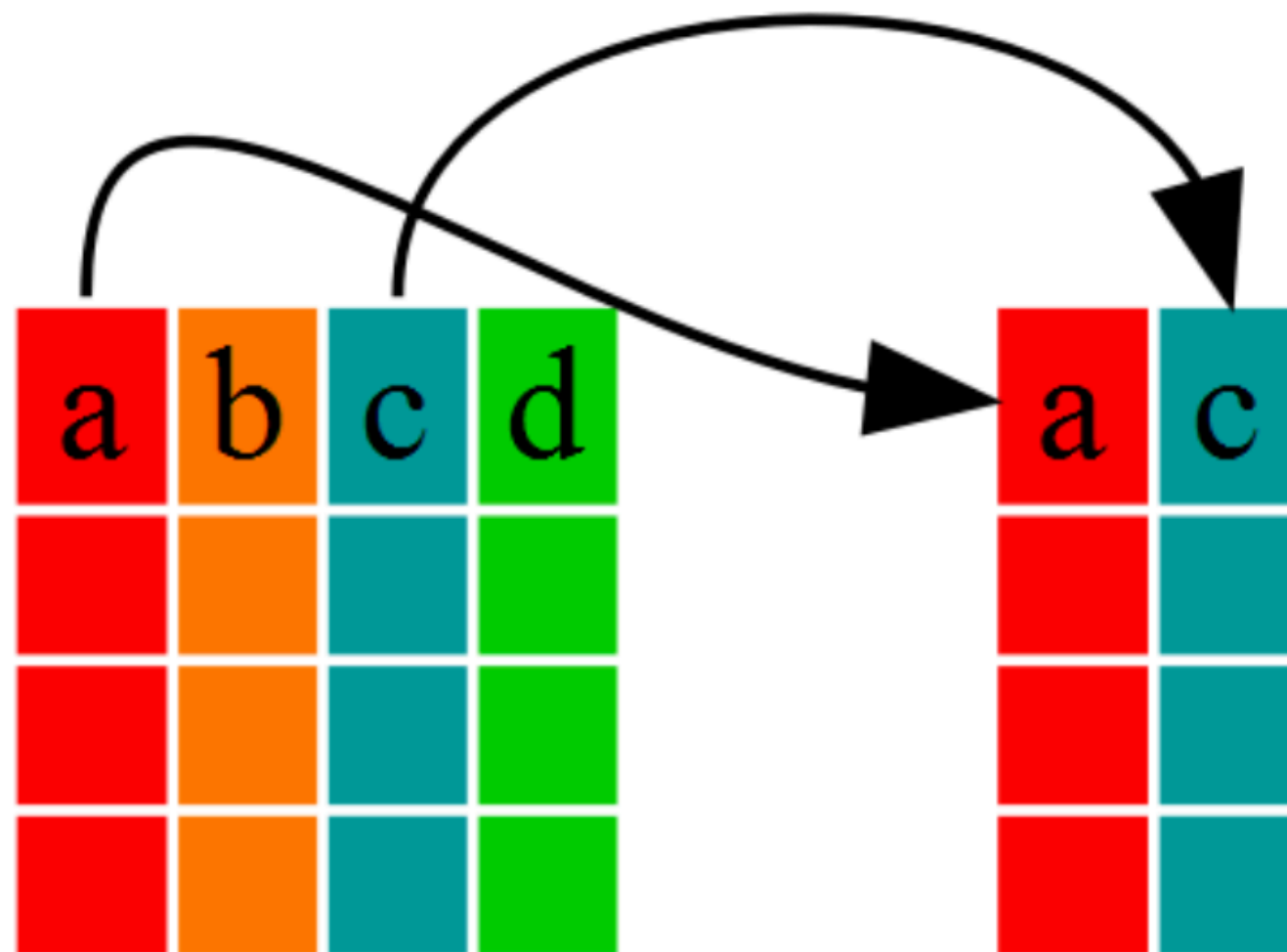


@allison_horst

select()



```
select(data.frame,a,c)
```



mutate()



arrange()



arrange()

storms

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21



storm	wind	pressure	date
Ana	40	1013	1997-07-01
Alex	45	1009	1998-07-30
Arthur	45	1010	1996-06-21
Arlene	50	1010	1999-06-13
Allison	65	1005	1995-06-04
Alberto	110	1007	2000-08-12

summarise()



city	particle size	amount ($\mu\text{g}/\text{m}^3$)
New York	large	23
New York	small	14



city	mean	sum	n
New York	18.5	37	2

London	large	22
London	small	16



London	19.0	38	2
--------	------	----	---

Beijing	large	121
Beijing	small	56



Beijing	88.5	177	2
---------	------	-----	---

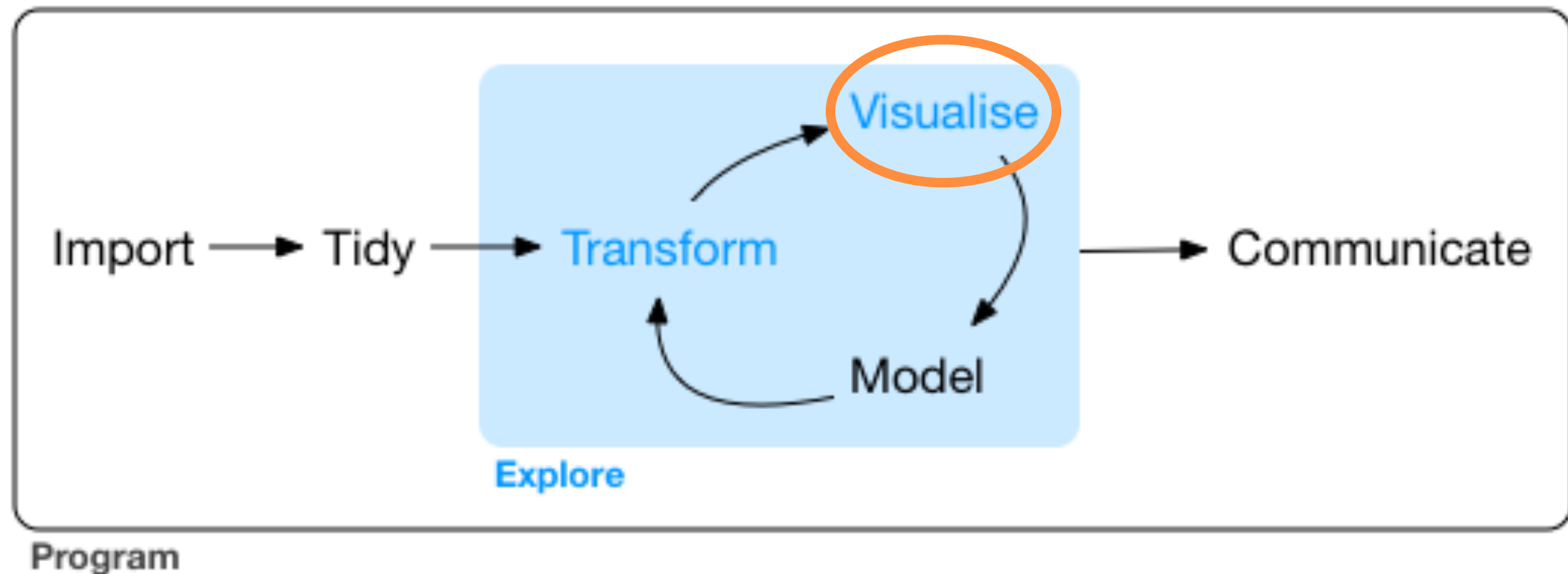
Visualización de datos en R



- * **El lenguaje de programación R provee librerías y herramientas para visualizar datos.** Las **ventajas** que ofrece realizar visualizaciones de R consisten en:
 - * **Alta personalización** de todos los elementos de nuestras gráficas.
 - * La posibilidad de trabajar **todo en un solo programa** (importación, exploración, limpieza, modelación, visualización y comunicación).
 - * La posibilidad de **generar visualizaciones interactivas** traduciendo código de R a HTML-CSS-JS y de generar aplicaciones web tipo shiny o documentos Markdown.

<https://www.data-to-viz.com/caveats.html>

Flujo de trabajo con datos



Hadley Wickham - R for data science
<https://r4ds.had.co.nz/explore-intro.html>

Paquetes de R para visualización



Los paquetes de R para visualización pueden incluir (mas no limitarse) a los siguientes:

Visualización estática:

- **base**: la librería base de R para hacer gráficas.
- **lattice**: librería que mejora las capacidades de r-base.
- **ggplot2**: la librería del tidyverse para hacer gráficas altamente personalizables.

Visualización interactiva:

- **plotly**, para gráficos interactivos básicos y compatible con ggplot2
- **Highcharter**, port de highcharts.js para R.
- **Amcharts**, visualización interactiva.
- **htmlwidgets**, familia de librerías para hacer visualización interactiva.
- **dygraphs**, para gráficas interactivas (especialmente Series de tiempo).
- **r2d3**, librería para hacer algunas gráficas de D3.js en R.

Mapas:

- **leaflet**: Mapas interactivos de leaflet.js
- También se pueden usar ggplot2, plotly o highcharter

Colores:

- **RColorBrewer**, paletas de colores.
- **viridis**, paletas de colores en R para personas con daltonismo.

Tablas:

- **DT**: Interface de la librería DataTable.js para R.
- **kableExtra**: Generar tablas con formato en HTML
- **formattable**: Tablas para presentación.
- **reactable**: Tablas con gráficas

Redes:

- **igraph**, para análisis y visualización básica de redes.
- **networkD3**, para redes interactivas y sankeys.

* Es la **librería del tidyverse más utilizada** para generar gráficas en R.

La programación de las gráficas requiere lo siguiente:

- 1) Un **objeto tibble/dataframe** (base de datos tabular) para darle la información.
- 2) **Iniciar el lienzo** con la función `ggplot()`
- 3) Definir los **elementos estéticos** (canales)
- 4) **Definir la(s) geometría(s)** o el tipo de gráfica a utilizar (marcas)
- 5) **Modificar elementos de la gráfica**, como etiquetas, formato de los ejes, sistema de coordenadas, límites de los valores, etc.
- 6) Opcional - **Definir temas y añadidos estéticos** para que la gráfica a se vea bonita.

1. Objeto tibble/dataframe



- 1) Un **objeto tibble/dataframe** (base de datos tabular) para darle la información.

```
> presion
# A tibble: 11 x 3
  systolicBloodPressure ageInYears weightInPounds
      <dbl>          <dbl>          <dbl>
1       132           52           173
2       143           59           184
3       153           67           194
4       162           73           211
5       154           64           196
6       168           74           220
7       137           54           188
8       149           61           188
9       159           65           207
10      128           46           167
11      166           72           217
```

2. Inicializar el lienzo

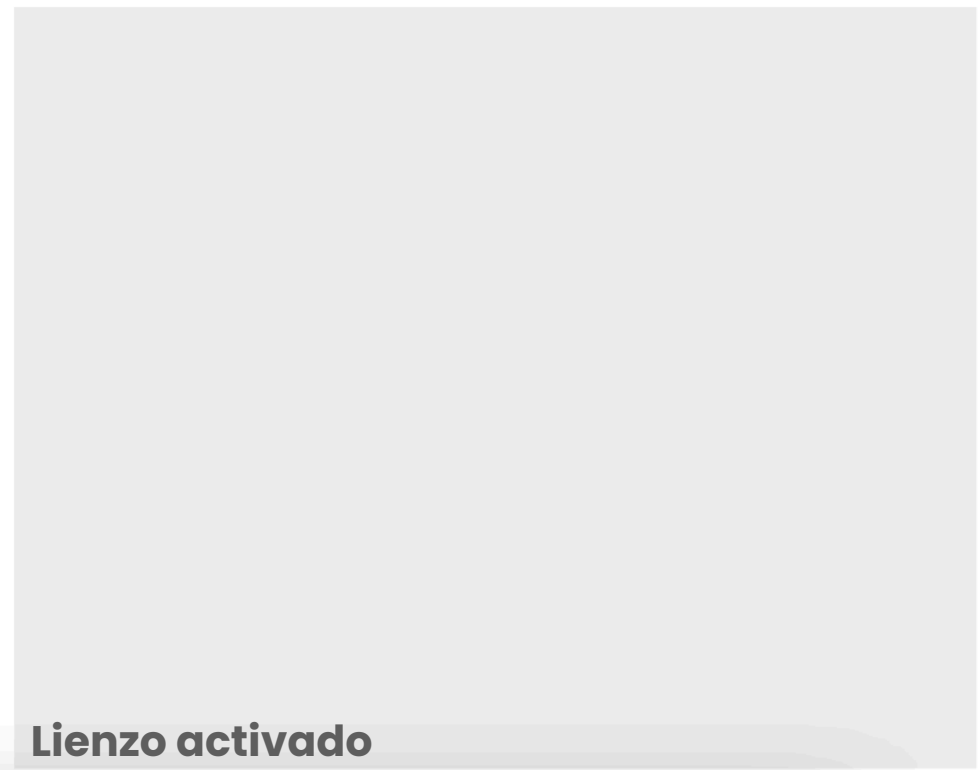


2) Inicializar el lienzo con la función ggplot()

- * Le pasamos el tibble del paso 1) a la función ggplot() para que se inicie el lienzo.
- * A partir de este punto las capas de la gráfica se van ligando con el símbolo de "+".
- * Dentro de ggplot() pueden irse incorporando los elementos estéticos.

```
presion %>%  
ggplot()
```

↑
Activar el lienzo



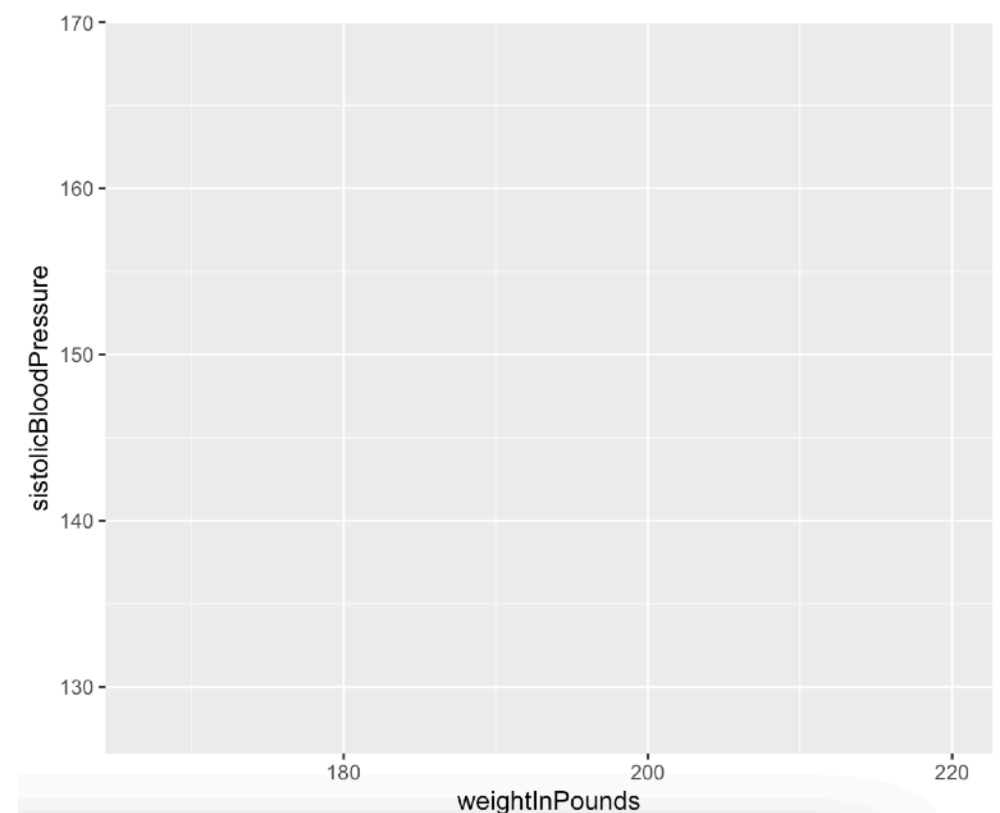
3. Definir los elementos estéticos



3) Definir los **elementos estéticos** (canales)

- * En este punto definimos las variables que van a definir la apariencia, no como la van a definir (esto se hace en un paso más adelante).
- * Los elementos estéticos a modificar son (pero no se limitan a) **la posición (coordenada x o y), el color, el relleno de una forma, el grosor de línea, el tamaño del punto, la transparencia, la forma que toma un punto o una línea, el grupo al que pertenece, etc.**

```
# Relación entre edad y presión sanguínea:  
presion %>%  
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y  
             x = weightInPounds,        # Posición eje X  
             color = ageInYears         # Color de punto  
          ))
```



4. Definir geometrías o marcas



4) Definir la geometría o el tipo de gráfica a utilizar (marcas)

- * En este paso se decide qué **geometría** utilizar.
- * Cada geometría **requiere los elementos estéticos mínimos para funcionar** (por ejemplo, si vas a usar puntos en 2D se requiere que al menos definas la coordenada x y y).
- * **Las geometrías más comunes se pueden consultar acá:**
 - * <https://ggplot2.tidyverse.org/reference/>

4. Definir geometrías o marcas



* Algunos ejemplos:

Columnas y barras:



```
geom_bar() geom_col()  
stat_count()
```

Bins y mapas de calor:



```
geom_bin_2d()  
stat_bin_2d()
```

Líneas rectas:



```
geom_abline() geom_hline()  
geom_vline()
```



```
geom_path() geom_line()  
geom_step()
```

Mapas:



```
coord_sf() geom_sf()  
geom_sf_label()  
geom_sf_text() stat_sf()
```

Puntos:



```
geom_jitter()  
geom_point()
```

Densidades:



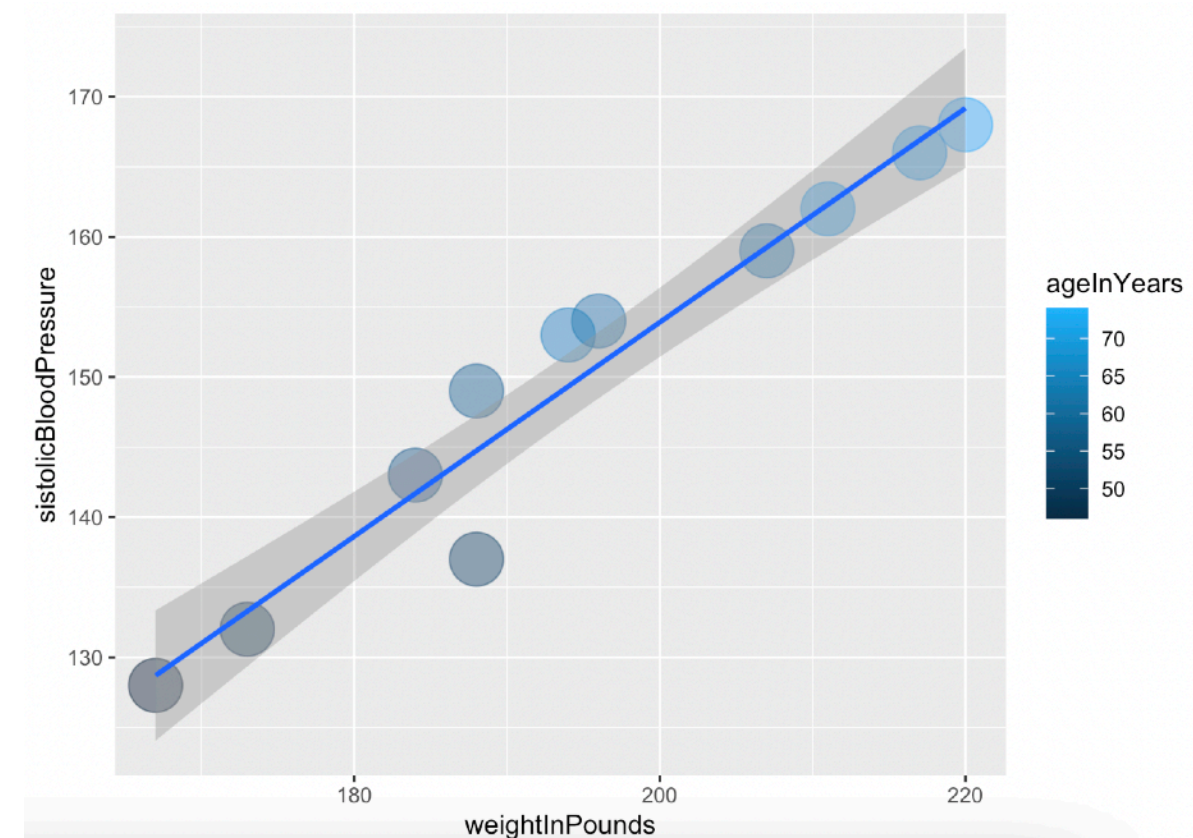
```
geom_boxplot()  
stat_boxplot()  
geom_density()  
stat_density()
```


4. Definir geometrías o marcas



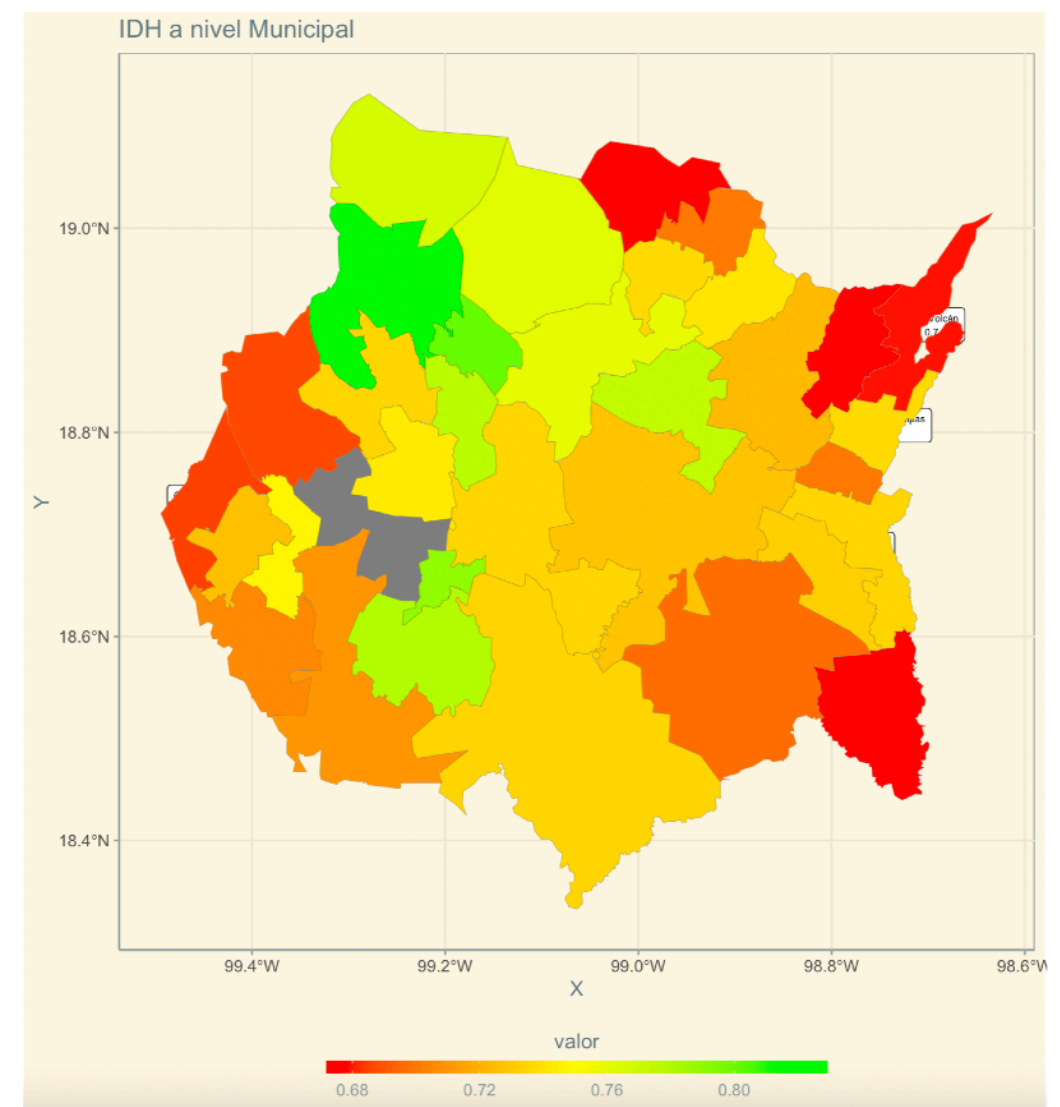
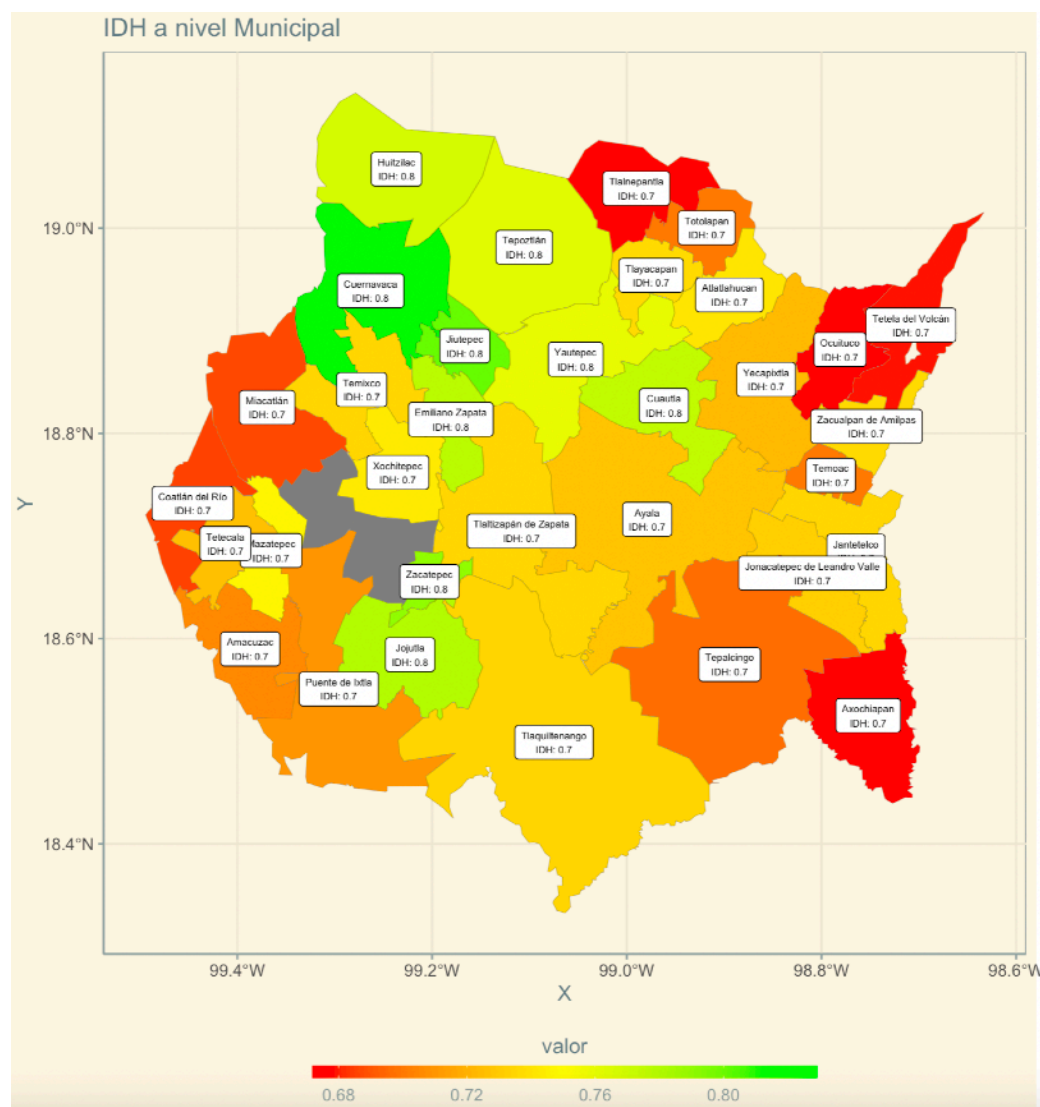
* Con código:

```
# Relación entre edad y presión sanguínea:
presion %>%
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y
             x = weightInPounds,        # Posición eje X
             color = ageInYears         # Color de punto
           )) +
  geom_point(size = 10, alpha = 0.5) + # geometria de puntos
  geom_smooth(method = "lm")           # líneas de regresión
```



The ggplot2 logo, which is a hexagon containing a line plot with blue dots and the text "ggplot2" below it.

* **Nota:** El orden importa. La geometría que pongas primero va “abajo” y la que se ponga después va arriba.



5. Modificar elementos de la gráfica



5) Modificar elementos de la gráfica, como etiquetas, formato de los ejes, sistema de coordenadas, límites de los valores, etc.

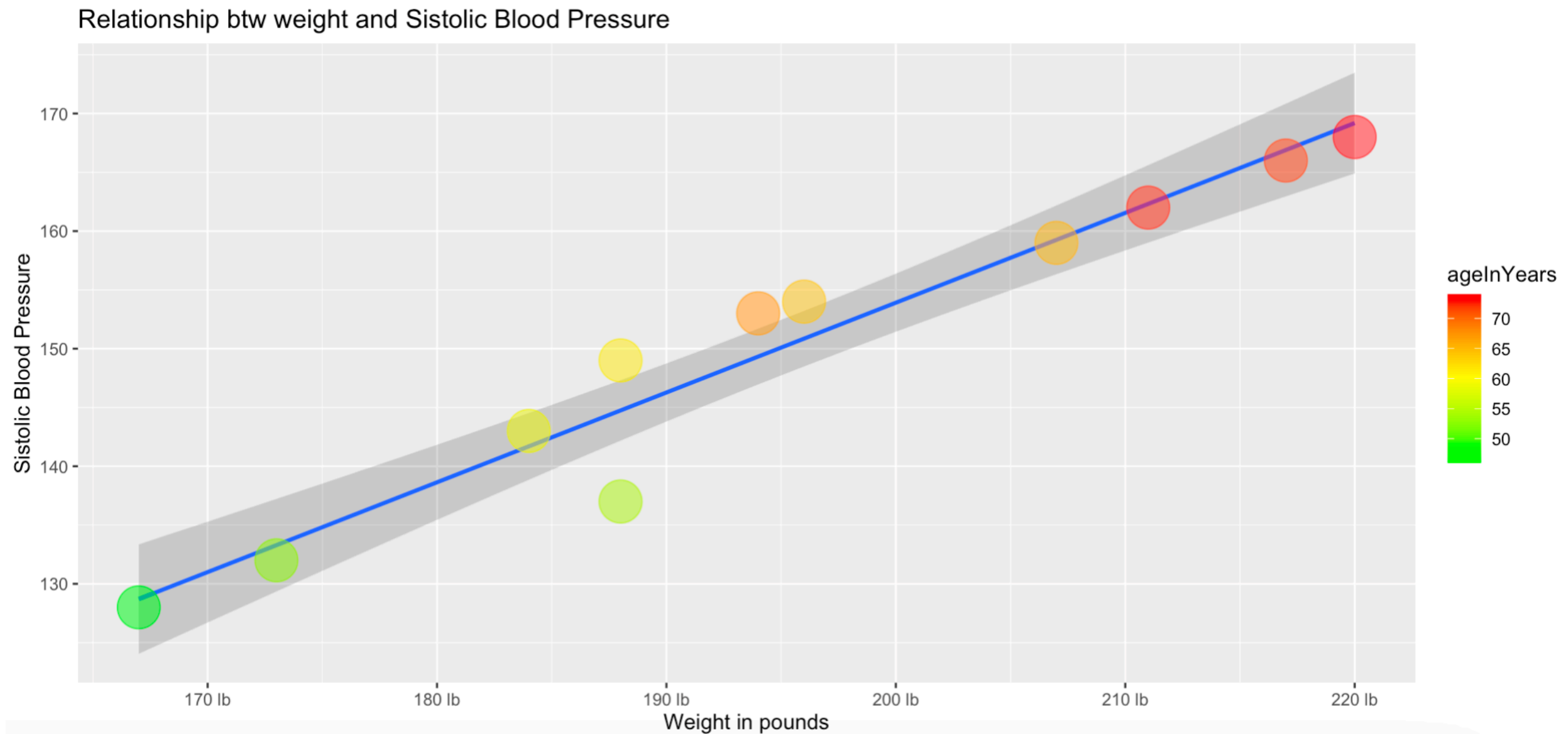
- * Para modificar los límites o los textos del eje x utilizamos las funciones `scale_x_*`()
- * Para modificar los colores, `scale_fill/color_*`()
- * Para añadir etiquetas, utilizamos la función `labs()` para modificar títulos, subtítulos, tags, pie de página, títulos de los ejes y de las leyendas.

5. Modificar elementos de la gráfica



```
# Relación entre edad y presión sanguínea:
presion %>%
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y
             x = weightInPounds,        # Posición eje X
             color = ageInYears         # Color de punto
           )) +
  geom_smooth(method = "lm") +          # líneas de regresión
  geom_point(size = 10, alpha = 0.5) + # geometría de puntos
  # Modificamos el eje X (si es numerico)
  scale_x_continuous(
    breaks = seq(from = 100,
                  to = 230,
                  by = 10), # Cada cuando se ponen las marcas
    labels = scales::comma_format(suffix = " lb") # Sobre las etiquetas
  ) +
  # Modificamos los colores de las geoms que usan el aes "color"
  scale_color_gradientn(colors = c("green", "yellow", "red")) +
  # Etiquetas de la gráfica
  labs(title = "Relationship btw weight and Systolic Blood Pressure",
        x = "Weight in pounds",
        y = "Systolic Blood Pressure",
        caption = "Source: Textbook data"
  )
```

5. Modificar elementos de la gráfica



The ggplot2 logo, which is a hexagon containing a line plot with blue dots and the text "ggplot2" below it.

The diagram illustrates the structure of a plot object and its components. It features a central plot area with data points and a legend on the right. Arrows indicate the relationships between various attributes and their corresponding visual elements.

Plot Components:

- Title:** The main title of the plot, associated with `plot.title`.
- Subtitle:** The subtitle of the plot, associated with `plot.subtitle`.
- Tag:** A label for the plot, associated with `plot.tag`.
- Color:** A legend for the plot, associated with `legend.title` and `legend.text`. It includes a color key (color, key, labels) and a shape key (shape, key, labels).
- Shape:** A legend for the plot, associated with `legend.title` and `legend.text`. It includes a color key (color, key, labels) and a shape key (shape, key, labels).

Axis Labels and Titles:

- y-axis label:** The label for the y-axis, associated with `axis.title.y` and `axis.title.left`.
- x-axis label:** The label for the x-axis, associated with `axis.title.x` and `axis.title.x.bottom`.
- axis.title:** The title for the axis, associated with `axis.title.y` and `axis.title.x`.
- axis.text:** The text for the axis, associated with `axis.text.y` and `axis.text.x`.
- axis.text.y.left:** The text for the y-axis, associated with `axis.text.y.left`.
- axis.text.x.bottom:** The text for the x-axis, associated with `axis.text.x.bottom`.

Plot Data and Legend:

- Color:** A legend for the plot, associated with `legend.title` and `legend.text`. It includes a color key (color, key, labels) and a shape key (shape, key, labels).
- Shape:** A legend for the plot, associated with `legend.title` and `legend.text`. It includes a color key (color, key, labels) and a shape key (shape, key, labels).

Plot Caption:

- Caption:** The caption for the plot, associated with `plot.caption`.

6.Temas: lo que se puede modificar



ggplot2 Theme Elements

`theme(element_name = element_function())`

- `element_text()`
- `element_line()`
- `element_rect()`
- `element_blank()`

Plot elements:

`plot.background`

`element_rect()`

`plot.title`

`element_text()`

`plot.margin`

`margin()`

Facetting elements:

`strip.background`

`element_rect()`

`panel.spacing`

`unit()`

`strip.text`

`element_text()`

Axis elements:

`axis.ticks`

`element_line()`

`axis.title`

`element_text()`

`axis.text`

`element_text()`

`axis.line`

`element_line()`

Legend elements:

`legend.margin`

`margin()`

`legend.title`

`element_text()`

`legend.key`

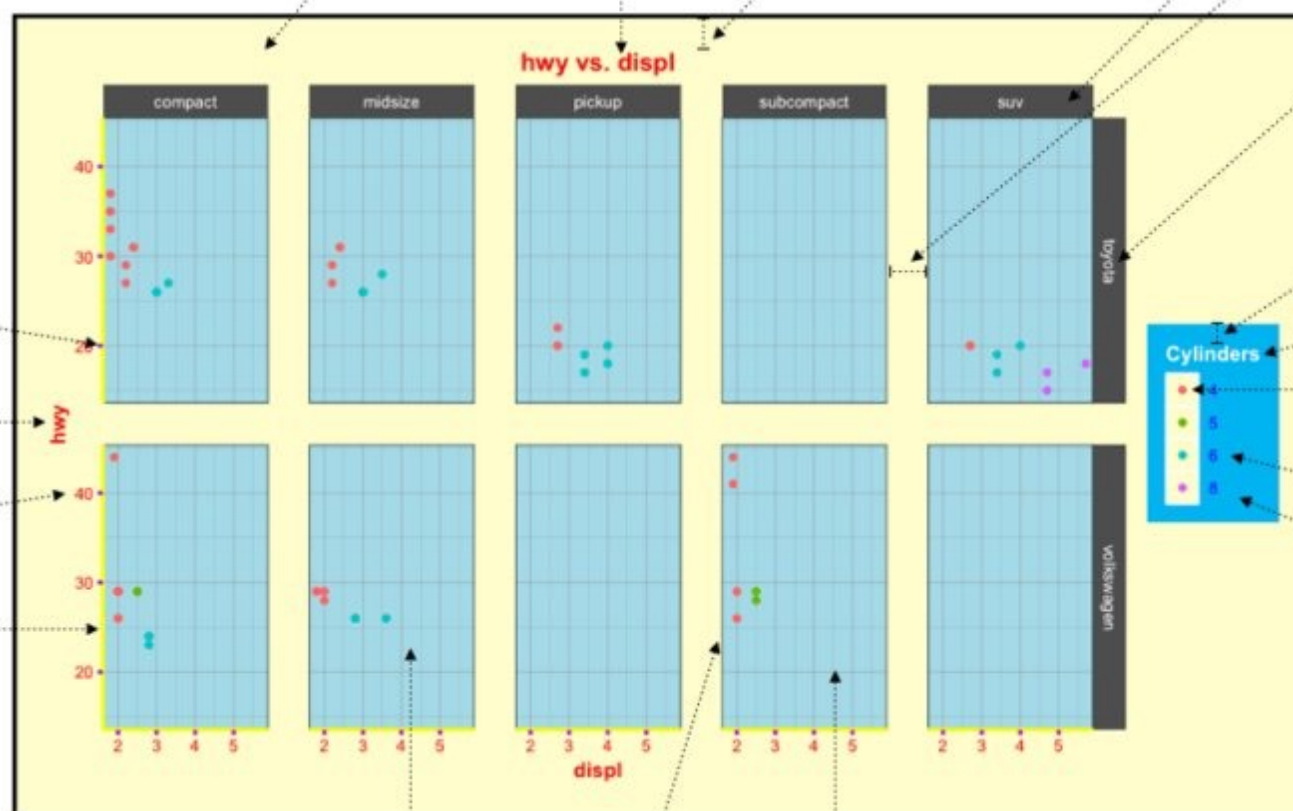
`element_rect()`

`legend.text`

`element_text()`

`legend.background`

`element_rect()`



`panel.background`

`element_rect()`

`panel.grid`

`element_line()`

`panel.border`

`element_rect(fill = NA)`

Panel elements:

henrywang.nl

Derived from "ggplot2: Elegant Graphics for Data Analysis"

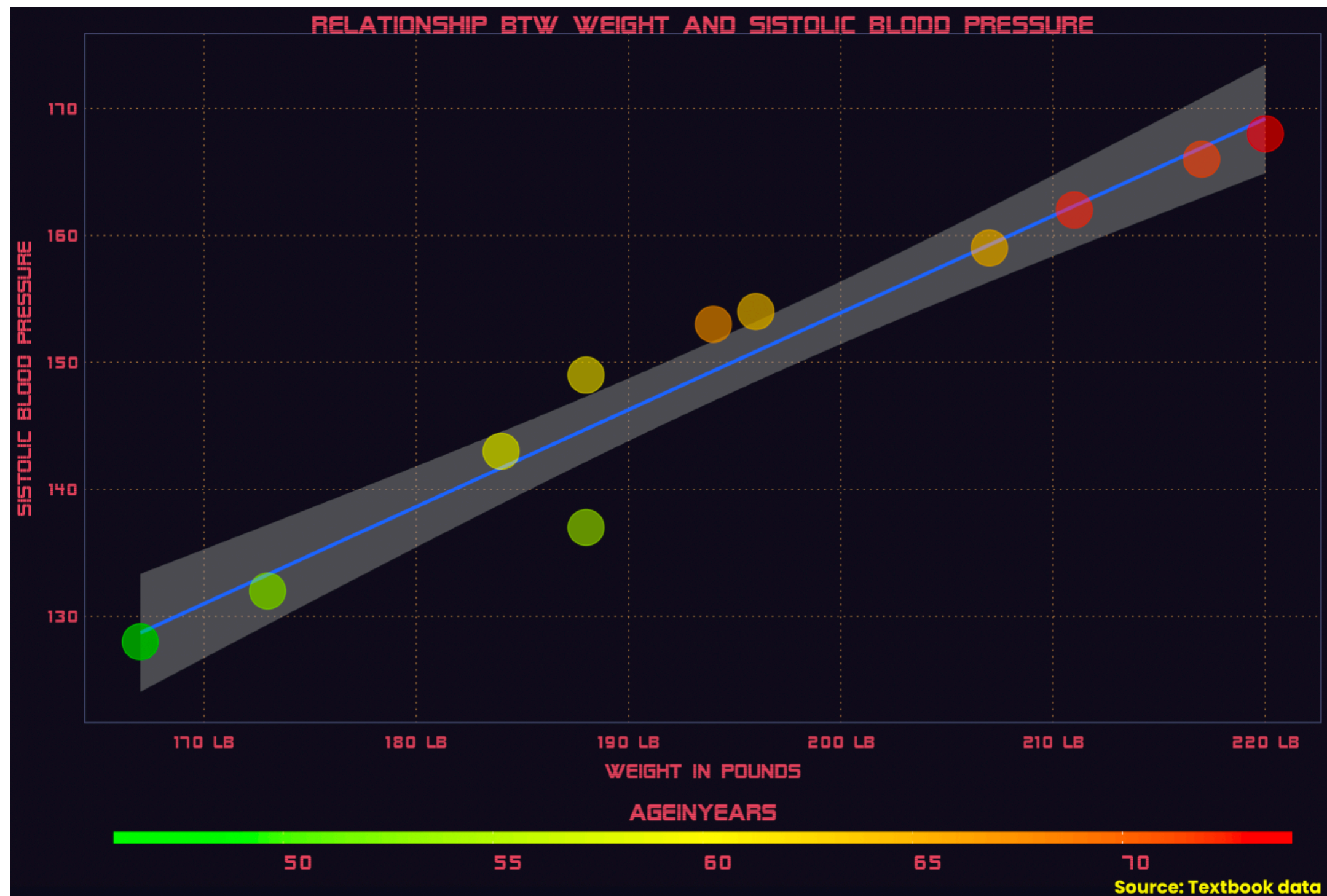
6.Temas: lo que se puede modificar



```
# Relación entre edad y presión sanguínea:
presion %>%
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y
             x = weightInPounds,        # Posición eje X
             color = ageInYears         # Color de punto
           )) +
  geom_smooth(method = "lm") +          # líneas de regresión
  geom_point(size = 10, alpha = 0.5) + # geometría de puntos
  # Modificamos el eje X (si es numerico)
  scale_x_continuous(
    breaks = seq(from = 100,
                  to = 230,
                  by = 10), # Cada cuando se ponen las marcas
    labels = scales::comma_format(suffix = " lb") # Sobre las etiquetas
  ) +
  # Modificamos los colores de las geoms que usan el aes "color"
  scale_color_gradientn(colors = c("green", "yellow", "red")) +
  # Etiquetas de la gráfica
  labs(title = "Relationship btw weight and Systolic Blood Pressure",
        x = "Weight in pounds",
        y = "Systolic Blood Pressure",
        caption = "Source: Textbook data") +
  vapoRwave::new_retro() +
  guides(color = guide_colorbar(title.position = "top",
                                title.hjust = 0.5,
                                barwidth = 50,
                                barheight = 0.5)) +
  theme(axis.title.y = element_text(angle = 90),
        legend.position = "bottom",
        plot.caption = element_text(hjust = 1,
                                     family = "Poppins",
                                     face = "bold",
                                     color = "yellow" ))
```

Podemos utilizar temas
pre-programados:

6.Temas: lo que se puede modificar



Tips



1. **Define bien tus colores** y tus paletas de colores.
2. **Ten a la mano los iconos que utilices más a menudo**, y dedícale el tiempo necesario a tener tu carpeta de íconos más utilizados (logos de tu empresa, de empresas con las que trabajes, partidos políticos, administraciones estatales, etc.)
3. Igualmente, **ten un repositorio de archivos que uses comúnmente**. Por ejemplo, para mapas yo tengo este repositorio de archivos: https://github.com/JuveCampos/Shapes_Resiliencia_CDMX_CIDE listos para crear mapas.
4. **Aprende como generar y subir páginas** en las cuales montar tus visualizaciones interactivas (Rmarkdown, Github Pages, shiny).
5. Trata de evitarlo, pero si no puedes, **hecha mano de la post-producción para editar tus gráficas** (por ejemplo, photoshop, illustrator o power point).

Más tips



1. Un buen ejercicio para empezar a practicar es el **replicar otras gráficas de otros programadores y usuarios**.
2. No te tienen que salir perfectas; recuerda que lo que te falte por hacer lo puedes corregir o añadir utilizando post-producción.
3. **Si estas en ceros en R**, te recomiendo que sepas: r-base, manejo de factores o variables categóricas, manejo de datos con el tidyverse, colores (rgb, hexadecimales) y pivoteo de bases (formato ancho y largo).
4. Si requieres **replicar una fuente** (tipo de letra) dada, usa la página <https://www.myfonts.com/WhatTheFont/> para guiarte y Google Fonts para descargar letras.

Ejercicio de procesamiento de datos

- 1. Cargue los datos de pobreza que se encuentran en su carpeta del ejercicio bajo el nombre ***df_pob_ent2.xlsx***.
- 2. Utilizando los verbos del tidyverse vistos, responda las siguientes preguntas:
 - ¿Cuántas columnas tiene? ¿Cuántas filas?
 - ¿Qué representa cada fila en la tabla?
 - ¿Cuál es el estado con mayor porcentaje de población de pobreza (pobreza) en 2024?
 - ¿Cuál es el estado con menor porcentaje de carencia por acceso a la salud?

Ejercicio de procesamiento de datos

- ¿Cuáles son los 10 estados con mayor porcentaje de pobreza extrema (pobreza_e) en 2024?
- ¿Cuál es el promedio simple del porcentaje de pobreza moderada (pobreza_m) de los estados del centro del país (CDMX, Puebla, EDOMEX, Morelos e Hidalgo) en 2024?
- ¿En qué año alcanzó Chiapas su porcentaje más alto de población con carencia por acceso a la seguridad social?
- ¿Cuál es el valor más bajo de pobreza que ha alcanzado algún estado (con los datos de la tabla)?
- Obtenga el promedio y la desviación estándar del porcentaje de pobreza para todos los estados, para cada año

Ejercicio de procesamiento de datos

- Genere una gráfica de barras para visualizar el porcentaje de población en situación de pobreza para cada entidad para el año 2024. Que las barras de los 10 estados con mayor porcentaje aparezcan en un tono de rojo, los 10 estados con menor porcentaje aparezcan en un tono de verde, y el resto que aparezcan en alguna variante de amarillo.
- Genere una gráfica para visualizar la evolución de la variable de Pobreza Extrema (pobreza_e) para el estado de Chiapas a lo largo de todos los años disponibles en la tabla. Guarde la gráfica en un archivo *.png
- **Extra:** Genere un bucle (con un loop for) para generar las gráficas de todos los demás estados de la república.

Gracias :3